

PE-EC801C — Error Correcting Codes

Semester VIII • 3L:0T:0P • 3 Credits

Module 5: Convolutional Codes and Decoding

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Introduction to Convolutional Codes	2
1.1	Core Parameters	2
1.2	Mathematical Representations	2
2	Structural Representations	3
2.1	State Transition Diagram	3
2.2	Tree Diagram	3
2.3	Trellis Diagram	4
3	Distance Properties of Convolutional Codes	4
3.1	Free Distance (d_{free})	4
3.2	Catastrophic Error Propagation	5
4	The Viterbi Decoding Algorithm	5
4.1	Algorithmic Steps	5
5	Sequential Decoding Algorithms	6
5.1	Wozencraft's Algorithm	6
5.2	Fano's Algorithm (Fann's Algorithm)	6
5.3	The Stack Algorithm (Zigangirov-Jelinek)	6
6	Worked Example of Viterbi Decoding	7
6.1	Step-by-Step Trellis Computations	7
6.2	Traceback Process	9

1. Introduction to Convolutional Codes

Unlike linear block codes, which segment the incoming message sequence into independent blocks of length k , **convolutional codes** process the message continuously. The coded output depends not only on the current input block but also on a finite number of previous input blocks stored in memory.

1.1 Core Parameters

A convolutional encoder is characterized by three main parameters:

1. **Code Rate (R):** Defined as $R = k/n$, where k input bits produce n output bits in each encoding cycle.
2. **Memory Stages (M):** The number of delay elements (flip-flops) in the shift registers.
3. **Constraint Length (K):** There are two standard definitions:
 - **System-Theoretic definition:** $K = M + 1$. This represents the maximum number of input cycles over which a single input bit can influence the encoder's output.
 - **Alternative definition:** $K = L \cdot k$, where L is the number of stages in the shift register.

1.2 Mathematical Representations

The encoding operation can be described in three algebraic formats:

Mathematical Formulations of Convolutional Coding

1. **Impulse Response:** Let $h^{(j)} = (h_0^{(j)}, h_1^{(j)}, \dots)$ be the impulse response of the path from the input to output j . For input sequence $u = (u_0, u_1, \dots)$, the output $v^{(j)}$ is the discrete convolution:

$$v_t^{(j)} = \sum_{i=0}^M u_{t-i} h_i^{(j)}$$

2. **Generator Matrix (G):** A semi-infinite, Block-Toeplitz matrix of the form:

$$G = \begin{bmatrix} G_0 & G_1 & G_2 & \dots & G_M & 0 & \dots \\ 0 & G_0 & G_1 & \dots & G_{M-1} & G_M & \dots \\ 0 & 0 & G_0 & \dots & G_{M-2} & G_{M-1} & \dots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots \end{bmatrix}$$

where each G_i is a $k \times n$ sub-matrix representing tap connections.

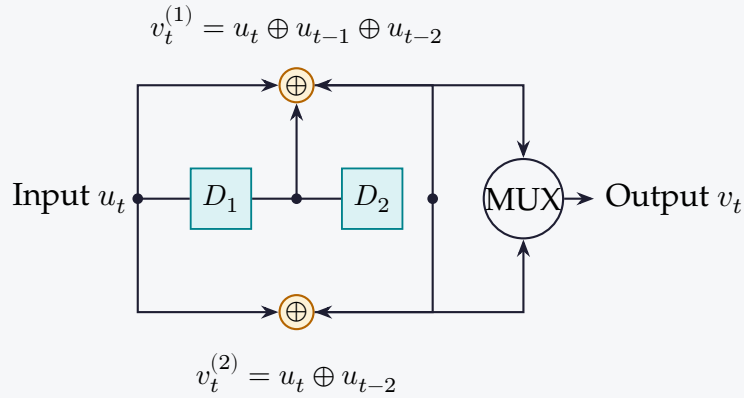
3. **Generator Polynomials:** Tap connections for the j -th output are expressed as a polynomial in the delay operator D :

$$g^{(j)}(D) = g_0^{(j)} \oplus g_1^{(j)} D \oplus g_2^{(j)} D^2 \oplus \dots \oplus g_M^{(j)} D^M$$

2. Structural Representations

Convolutional encoders are implemented using feed-forward shift registers and modulo-2 adders. They can be analyzed using three graphical models.

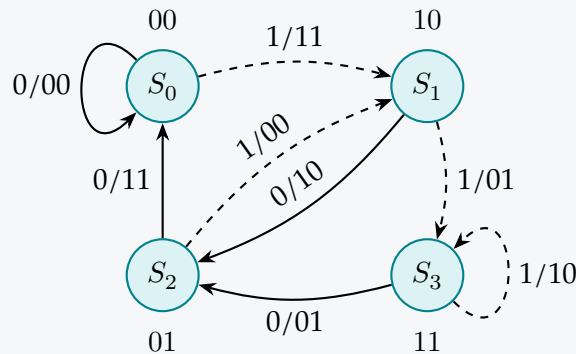
Feed-Forward Encoder for a Rate 1/2 ($K = 3$) Convolutional Code



2.1 State Transition Diagram

A state transition diagram represents the encoder as a finite-state machine. The state is determined by the previous bits stored in the shift register: $S_t = (u_{t-1}, u_{t-2}, \dots, u_{t-M})$. For $M = 2$, there are four possible states: 00, 10, 01, and 11.

State Transition Diagram (4-State, $K=3$ Encoder)

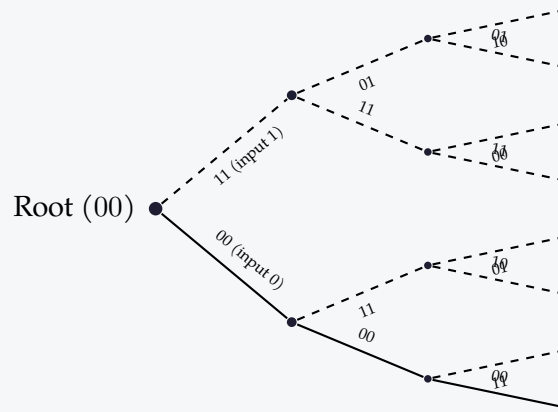


Legend: Solid lines represent transitions caused by input '0'. Dashed lines represent transitions caused by input '1'. Labels indicate Input/Output.

2.2 Tree Diagram

The tree diagram represents all possible output paths starting from the initial state 00. For each incoming bit, the path splits into two branches: the upper branch represents input 0, and the lower branch represents input 1. The diagram grows exponentially, making it suitable only for illustrating short sequences.

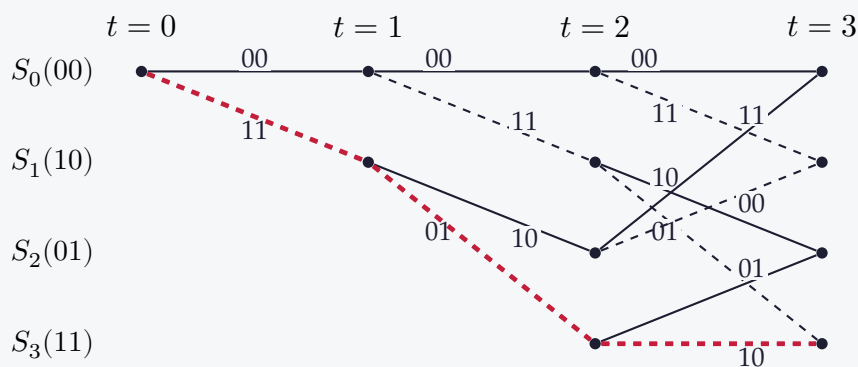
3-Level Tree Diagram Representation



2.3 Trellis Diagram

By merging node branches corresponding to identical states in the tree diagram, we obtain the **trellis diagram**. The trellis diagram is a time-indexed state diagram. Since the number of states is fixed (2^M), the trellis width is constant for all time intervals $t \geq M$, providing a highly compact layout for analyzing long sequences.

Trellis Diagram Section showing State Transitions and Survivor Path



3. Distance Properties of Convolutional Codes

The error-correcting capability of a convolutional code depends on its distance properties.

3.1 Free Distance (d_{free})

The most important distance metric is the **free distance** d_{free} .

Free Distance Definition

The free distance d_{free} is the minimum Hamming distance between any two distinct code sequences in the trellis that begin at state 00 and eventually diverge, only to merge back at state 00 at some later step.

$$d_{\text{free}} = \min \{d_H(v^{(1)}, v^{(2)}) \mid u^{(1)} \neq u^{(2)}\}$$

If we set one sequence to be the all-zero codeword, d_{free} simplifies to the minimum weight of a non-zero code sequence starting and ending at state 00. The number of errors t that the code can correct is:

$$t = \left\lfloor \frac{d_{\text{free}} - 1}{2} \right\rfloor$$

3.2 Catastrophic Error Propagation

Catastrophic error propagation is a key hazard in convolutional code design.

Catastrophic Error Propagation

A code is **catastrophic** if a finite number of channel transmission errors leads to an infinite number of decoding decisions/bit errors in the reconstructed message sequence.

Sufficient Condition: In the state transition diagram of a catastrophic code, there exists a closed loop (cycle) containing a state other than 00 where all transitions along the loop produce zero-weight output bits (all output bits are '0'), despite being triggered by non-zero inputs. For systematic convolutional codes, catastrophic propagation is impossible because the inputs are mirrored directly in the output stream, preventing zero-weight outputs for non-zero inputs.

4. The Viterbi Decoding Algorithm

The **Viterbi algorithm** is a Maximum Likelihood (ML) decoding technique that minimizes the probability of sequence error. It systematically searches the code trellis to find the path that is closest to the received vector.

4.1 Algorithmic Steps

Let the received sequence be $r = (r_0, r_1, \dots, r_{L-1})$. At each time step t , the decoder maintains a single survivor path for each state, along with its accumulated path metric:

1. **Branch Metric (BM):** Compute the distance between the received segment r_t and the expected output on each branch:
 - **Hard Decision:** Hamming distance $d_H(r_t, v_{\text{branch}})$.
 - **Soft Decision:** Euclidean distance $\|r_t - v_{\text{branch}}\|^2$.
2. **Path Metric (PM):** Calculate candidates for each state:

$$PM_t(S) = \min_{S' \rightarrow S} [PM_{t-1}(S') + BM(S' \rightarrow S)]$$

3. **Add-Compare-Select (ACS):** For each state S at step t :

- **Add** branch metrics to the previous path metrics of all incoming states S' .
 - **Compare** the candidate path metrics.
 - **Select** the path with the minimum metric as the survivor. Discard all other incoming paths.
4. **Traceback:** Once the end of the trellis is reached (or after a fixed decoding depth), trace the path backward from the state with the lowest path metric to recover the input sequence.

5. Sequential Decoding Algorithms

Sequential decoding algorithms are sub-optimal, tree-based decoding techniques. Unlike Viterbi, which searches all trellis paths in parallel, sequential decoders explore only one path at a time. They are suitable for codes with large constraint lengths K , where Viterbi's complexity (2^{K-1}) is prohibitive.

5.1 Wozencraft's Algorithm

The pioneer sequential decoding algorithm, Wozencraft's algorithm, performs a depth-first search on the code tree. It traces a path forward. If the accumulated path metric exceeds a threshold (indicating too many errors), the decoder halts, backs up (backtracks), and attempts to explore alternative branches.

5.2 Fano's Algorithm (Fann's Algorithm)

Fano's algorithm (referred to as Fann's in some syllabi) is a refinement of tree search that operates without a storage stack:

- It uses a variable threshold that is dynamically adjusted based on channel quality.
- The decoder moves forward along the branch that maximizes the Fano metric (a metric that penalizes paths that deviate from typical channel characteristics).
- If the path metric falls below the threshold, the decoder lowers the threshold and backs up to explore other paths.
- **Advantage:** Low memory requirements. **Disadvantage:** Variable decoding time depending on channel noise.

5.3 The Stack Algorithm (Zigangirov-Jelinek)

The Stack (or priority queue) algorithm is a fast sequential search method:

- The decoder maintains a list (stack) of explored paths of varying lengths, ordered by their path metrics.
- In each cycle, the decoder removes the path with the best metric from the top of the stack.
- It extends this path by computing metrics for its two child branches and inserts the new paths back into the stack.
- The stack is re-sorted, and the process repeats until a path reaches the end of the tree.

- **Advantage:** Eliminates backtracking. **Disadvantage:** Requires a large memory space to maintain and sort the stack.

6. Worked Example of Viterbi Decoding

Consider the $(2, 1, 2)$ convolutional code shown in Section 2. The encoder starts at state $S_0(00)$. We receive the sequence $r = (11, 01, 01, 00)$. Let's decode it using the Viterbi algorithm.

6.1 Step-by-Step Trellis Computations

Step 1: Time $t = 1$ (Received $r_0 = 11$)

The only active starting state is $S_0(00)$ with path metric $PM_0(S_0) = 0$.

- Transition $S_0 \rightarrow S_0$ (input 0): output 00. $BM = d_H(11, 00) = 2$. $PM_1(S_0) = PM_0(S_0) + 2 = 2$. (Survivor: $S_0 \rightarrow S_0$)
- Transition $S_0 \rightarrow S_1$ (input 1): output 11. $BM = d_H(11, 11) = 0$. $PM_1(S_1) = PM_0(S_0) + 0 = 0$. (Survivor: $S_0 \rightarrow S_1$)

States S_2 and S_3 are unreachable at $t = 1$ ($PM_1 = \infty$).

Step 2: Time $t = 2$ (Received $r_1 = 01$)

We transition from active states S_0 ($PM_1 = 2$) and S_1 ($PM_1 = 0$):

- From S_0 :
 - To S_0 (input 0, output 00): $BM = d_H(01, 00) = 1$. Candidate $PM = 2 + 1 = 3$.
 - To S_1 (input 1, output 11): $BM = d_H(01, 11) = 1$. Candidate $PM = 2 + 1 = 3$.
- From S_1 :
 - To S_2 (input 0, output 10): $BM = d_H(01, 10) = 2$. Candidate $PM = 0 + 2 = 2$.
 - To S_3 (input 1, output 01): $BM = d_H(01, 01) = 0$. Candidate $PM = 0 + 0 = 0$.

Since there are no collisions yet, the survivors at $t = 2$ are: $PM_2(S_0) = 3$, $PM_2(S_1) = 3$, $PM_2(S_2) = 2$, and $PM_2(S_3) = 0$.

Step 3: Time $t = 3$ (Received $r_2 = 01$)

All states are active. We apply the Add-Compare-Select (ACS) operation for each state:

- **State $S_0(00)$:**
 - From S_0 ($PM = 3$): branch output 00, $BM = d_H(01, 00) = 1$. Path candidate = $3 + 1 = 4$.
 - From S_2 ($PM = 2$): branch output 11, $BM = d_H(01, 11) = 1$. Path candidate = $2 + 1 = 3$.
 - **Select:** $PM_3(S_0) = \min(4, 3) = 3$. (Survivor path came from S_2).
- **State $S_1(10)$:**

- From S_0 ($PM = 3$): branch output 11, $BM = d_H(01, 11) = 1$. Path candidate = $3 + 1 = 4$.
- From S_2 ($PM = 2$): branch output 00, $BM = d_H(01, 00) = 1$. Path candidate = $2 + 1 = 3$.
- **Select:** $PM_3(S_1) = \min(4, 3) = 3$. (Survivor path came from S_2).
- **State $S_2(01)$:**
 - From S_1 ($PM = 3$): branch output 10, $BM = d_H(01, 10) = 2$. Path candidate = $3 + 2 = 5$.
 - From S_3 ($PM = 0$): branch output 01, $BM = d_H(01, 01) = 0$. Path candidate = $0 + 0 = 0$.
 - **Select:** $PM_3(S_2) = \min(5, 0) = 0$. (Survivor path came from S_3).
- **State $S_3(11)$:**
 - From S_1 ($PM = 3$): branch output 01, $BM = d_H(01, 01) = 0$. Path candidate = $3 + 0 = 3$.
 - From S_3 ($PM = 0$): branch output 10, $BM = d_H(01, 10) = 2$. Path candidate = $0 + 2 = 2$.
 - **Select:** $PM_3(S_3) = \min(3, 2) = 2$. (Survivor path came from S_3).

Step 4: Time $t = 4$ (Received $r_3 = 00$)

We perform ACS for the final time step:

- **State $S_0(00)$:**
 - From S_0 ($PM_3 = 3$): branch output 00, $BM = d_H(00, 00) = 0$. Path candidate = $3 + 0 = 3$.
 - From S_2 ($PM_3 = 0$): branch output 11, $BM = d_H(00, 11) = 2$. Path candidate = $0 + 2 = 2$.
 - **Select:** $PM_4(S_0) = \min(3, 2) = 2$. (Survivor path came from S_2).
- **State $S_1(10)$:**
 - From S_0 ($PM_3 = 3$): branch output 11, $BM = d_H(00, 11) = 2$. Path candidate = $3 + 2 = 5$.
 - From S_2 ($PM_3 = 0$): branch output 00, $BM = d_H(00, 00) = 0$. Path candidate = $0 + 0 = 0$.
 - **Select:** $PM_4(S_1) = \min(5, 0) = 0$. (Survivor path came from S_2).
- **State $S_2(01)$:**
 - From S_1 ($PM_3 = 3$): branch output 10, $BM = d_H(00, 10) = 1$. Path candidate = $3 + 1 = 4$.
 - From S_3 ($PM_3 = 2$): branch output 01, $BM = d_H(00, 01) = 1$. Path candidate = $2 + 1 = 3$.
 - **Select:** $PM_4(S_2) = \min(4, 3) = 3$. (Survivor path came from S_3).
- **State $S_3(11)$:**

- From S_1 ($PM_3 = 3$): branch output 01, $BM = d_H(00, 01) = 1$. Path candidate = $3 + 1 = 4$.
- From S_3 ($PM_3 = 2$): branch output 10, $BM = d_H(00, 10) = 1$. Path candidate = $2 + 1 = 3$.
- **Select:** $PM_4(S_3) = \min(4, 3) = 3$. (Survivor path came from S_3).

6.2 Traceback Process

At the end of the trellis ($t = 4$), the state with the minimum path metric is S_1 with $PM_4(S_1) = 0$. We trace this path backward:

1. At $t = 4$, we are in state $S_1(10)$. The survivor came from state $S_2(01)$ at $t = 3$. The transition $S_2(01) \rightarrow S_1(10)$ is triggered by input bit **1**.
2. At $t = 3$, we are in state $S_2(01)$. The survivor came from state $S_3(11)$ at $t = 2$. The transition $S_3(11) \rightarrow S_2(01)$ is triggered by input bit **0**.
3. At $t = 2$, we are in state $S_3(11)$. The survivor came from state $S_1(10)$ at $t = 1$. The transition $S_1(10) \rightarrow S_3(11)$ is triggered by input bit **1**.
4. At $t = 1$, we are in state $S_1(10)$. The survivor came from state $S_0(00)$ at $t = 0$. The transition $S_0(00) \rightarrow S_1(10)$ is triggered by input bit **1**.

Thus, the decoded message sequence is $u = (1, 1, 0, 1)$, and the transmitted codeword is $c = (11, 01, 01, 00)$. Since $r = c$, the accumulated Hamming distance is indeed 0.

Quick Revision — Module 5

Convolutional Codes and Decoding | PE-EC801C

Key Formulas and Metrics at a Glance

Code Rate	$R = \frac{k}{n}$
Constraint Length (Standard)	$K = M + 1$ (M is the number of delay stages)
Impulse Response Convolution	$v_t^{(j)} = \sum_{i=0}^M u_{t-i} h_i^{(j)}$
Discrete Delay Polynomial	$g^{(j)}(D) = g_0^{(j)} \oplus g_1^{(j)} D \oplus \dots \oplus g_M^{(j)} D^M$
Error Correction Power	$t = \left\lfloor \frac{d_{\text{free}} - 1}{2} \right\rfloor$
Viterbi Path Metric Update	$PM_t(S) = \min_{S' \rightarrow S} [PM_{t-1}(S') + BM(S' \rightarrow S)]$

One-Line Definitions

- **Convolutional Code:** A continuous coding technique where outputs are generated by convolving current and past message bits.
- **Constraint Length (K):** The number of input bits that reside in the encoder and influence the output.
- **Trellis Diagram:** A time-indexed state diagram showing all possible transitions over time.
- **Free Distance (d_{free}):** The minimum Hamming distance between any two distinct paths in the trellis starting and ending in 00.
- **Catastrophic Code:** A code where a finite number of channel errors causes an infinite number of decoded bit errors.
- **Survivor Path:** The path entering a trellis state that has the minimum accumulated path metric.
- **ACS (Add-Compare-Select):** The core hardware operation in Viterbi decoding that updates path metrics and retains survivors.
- **Fano Metric:** A dynamic search metric that penalizes paths that deviate from typical channel behavior.

Frequently Asked / Viva Questions

Q1. What is the difference between block codes and convolutional codes?

Linear block codes process messages in independent block segments, and the current output block depends only on the current input block. Convolutional codes process the input continuously, and the output depends on the current input and a finite memory of past inputs.

Q2. Define constraint length K in a convolutional encoder.

Constraint length K represents the number of input cycles over which an input bit remains in the shift register and influences the output. It is mathematically defined as $K = M + 1$, where M is the number of memory delay elements.

Q3. What is free distance d_{free} and why is it important?

Free distance d_{free} is the minimum Hamming distance between any two paths in the trellis that diverge from the all-zero state and merge back later. It determines the code's error-correcting capability: $t = \lfloor (d_{\text{free}} - 1)/2 \rfloor$.

Q4. How do you identify a catastrophic convolutional code from its state diagram?

A code is catastrophic if its state transition diagram contains a loop of non-zero states where all outputs are zero. If transmission errors cause the decoder to enter this loop, it will continue to output errors indefinitely.

Q5. State the three structural representations of convolutional codes.

The three structural representations are the State Transition Diagram (FSM representation), the Tree Diagram (exponential tree branching showing all output sequences), and the Trellis Diagram (compact time-aligned state transitions).

Q6. Explain the ACS operation in the Viterbi algorithm.

ACS stands for Add-Compare-Select. For each trellis state, it adds the branch metrics of incoming branches to the path metrics of their origin states, compares the candidate values, and selects the path with the minimum metric as the survivor.

Q7. What is the difference between hard-decision and soft-decision decoding?

Hard-decision decoding processes demodulated binary data ('0' or '1') and uses Hamming distance. Soft-decision decoding processes unquantized analog voltage values directly from the channel and uses Euclidean distance, offering a 2–3 dB coding gain.

Q8. Compare Viterbi decoding with sequential decoding.

Viterbi decoding is a maximum likelihood algorithm that searches all trellis paths in parallel, requiring 2^M state processors. Sequential decoding explores a single path at a time in a tree structure, using backtracking or stacks, which reduces memory but introduces variable decoding delays.

Q9. How does Fano's sequential decoding algorithm work?

Frequently Asked / Viva Questions (contd.)

Fano's algorithm performs a tree search by moving forward or backward along branches based on a metric threshold. If the path metric falls below the threshold, it lowers the threshold and backtracks, requiring no memory stacks.

Q10. What is the Stack algorithm, and what is its main drawback?

The Stack algorithm (Zigangirov-Jelinek) maintains a priority queue of paths ordered by their metrics, extending the top path. Its main drawback is the large memory required to store and sort the paths, and the risk of stack overflow under noisy conditions.

Structural Comparison of Block and Convolutional Codes

Property	Linear Block Codes	Convolutional Codes
Encoding Mode	Block-by-block (independent segments)	Continuous stream processing
Memory Structure	Memoryless (outputs depend on current block)	Finite memory (outputs convolved with history)
Key Parameters	Code length n , message length k	Rate k/n , constraint length K , memory M
Decoding Algorithm	Syndrom table lookup / Meggitt decoding	Viterbi / Sequential decoding

Comparison of Viterbi vs. Sequential Decoding

Property	Viterbi Decoding	Sequential Decoding
Search Strategy	Parallel search over all trellis paths	Single-path depth-first tree search
Optimality	Maximum Likelihood (optimal)	Sub-optimal
Decoding Time	Constant (independent of channel noise)	Variable (increases with channel noise)
Constraint Limit	Limited to small K ($K \leq 9$)	Can decode large K ($K \geq 20$)
Memory Demand	Low storage (only survivor paths)	High storage (stack) or backtracking overhead